

Overcoming the Retrieval Barrier : Indirect Prompt Injection in the Wild for LLM Systems

Sadman Bin Tareq, 2105040
Rageeb Hasan Shafee, 2105175

Paper Overview

Paper Topic:

Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems

Main Problem:

Indirect Prompt Injection attacks depend on malicious external documents reaching the LLM context.

Modern LLM systems retrieve information from:

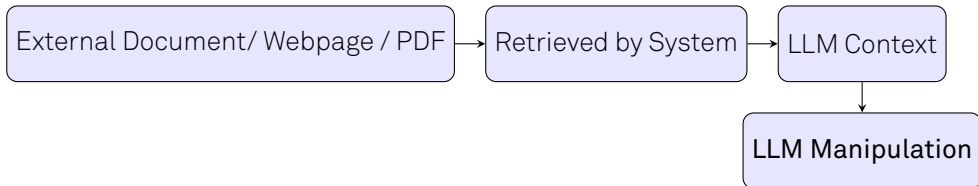
- Emails
- Documents
- Web pages
- Knowledge bases
- Databases

Central Idea

A malicious document cannot influence an LLM unless it is first successfully retrieved.

What is Indirect Prompt Injection?

Indirect Prompt Injection occurs when malicious instructions are stored inside external data instead of being directly entered by the user.



Important

The attacker does not directly communicate with the LLM. The malicious instruction reaches the LLM indirectly through retrieved data.

The Retrieval Barrier

Previous IPI research often assumes:

Malicious Document → Already Inside LLM Context

However, real retrieval systems work differently:

Malicious Document → Retrieval Competition → Possibly Retrieved → LLM

Central Question

How can an attacker ensure that a malicious document is retrieved under natural user queries?

This problem is called the Retrieval Barrier

Why Retrieval is the Critical Bottleneck

For an indirect prompt injection attack:

No Retrieval $\Rightarrow$ No Prompt Injection

A normal malicious instruction may fail because legitimate documents are more semantically relevant to the user's query.

Core Insight

The challenge is not only creating a malicious instruction.
The malicious document must first win the retrieval competition.

Therefore:

Retrieval Security is an Important Part of LLM Security

Key Attack Idea

The malicious document is divided into two components:

$$x \parallel D_{adv}$$

Trigger Fragment (x)

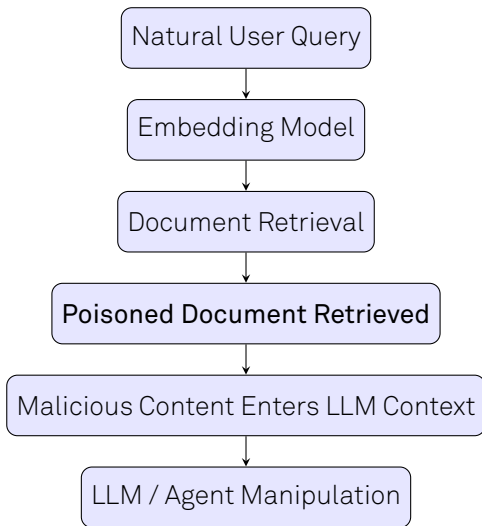
- Optimized for retrieval.
- Increases embedding similarity.
- Helps the malicious document reach Top- K results.

Attack Fragment (D_{adv})

- Contains malicious instructions.
- Manipulates the LLM after retrieval.
- Determines the final attack objective.

Optimized Trigger + Attack Instruction

Overall Attack Pipeline



Trigger Optimization Objective

The attacker searches for a trigger fragment x that maximizes similarity between the query and malicious document.

$$f(x) = \text{sim}(E(q), E(x \parallel D_{adv}))$$

where:

- q = user query
- x = optimized trigger fragment
- D_{adv} = malicious attack fragment
- E = embedding model

The goal is:

$$x \parallel D_{adv} \rightarrow \text{Top-K Retrieved Documents}$$

Cross-Entropy Method (CEM)

The paper uses the:

Cross-Entropy Method (CEM)

to construct optimized trigger fragments.

Why is this important?

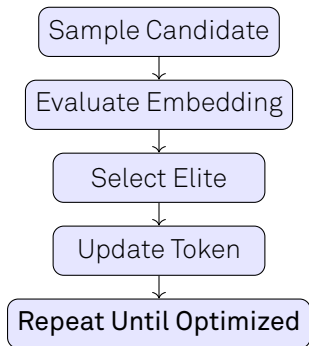
The attack does not require access to:

- Embedding model parameters
- Gradients
- Target LLM parameters
- Complete retrieval corpus

Threat Model

The attacker can optimize triggers using black-box access to an embedding model.

How the CEM Attack Works



Experimental Evaluation

The paper evaluates the approach across:

- 11 information retrieval benchmarks
- 8 embedding models
- Open-source embedding models
- Proprietary embedding APIs
- RAG systems
- Single-agent systems
- Multi-agent systems

Comparison:

1. Vanilla malicious attack fragment
2. Query+ baseline
3. CEM-optimized trigger

Experimental Evaluation

Main Finding

Optimized trigger fragments significantly improve the probability that malicious documents are retrieved.

Key Understanding 1: Retrieval is the Bottleneck

The most important question is:

Can the malicious document actually be retrieved?

The paper changes the threat model from:

Malicious Instruction $\rightarrow$ LLM

to:

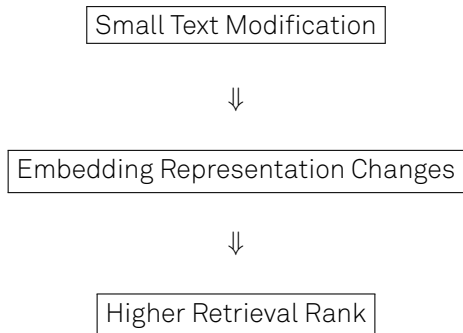
Malicious Document $\rightarrow$ Retrieval $\rightarrow$ LLM

Main Contribution

The paper identifies retrieval as the missing link between malicious external data and successful indirect prompt injection.

Key Understanding 2: Small Trigger, Large Effect

A relatively short optimized trigger can significantly affect document ranking.



Security Implication

A document can manipulate retrieval behavior without visibly looking like the user's query.

Key Understanding 3: Better Models not Automatically Safer

The paper evaluates multiple embedding models with different:

- Architectures
- Model sizes
- Providers
- Open-source implementations
- Proprietary implementations

Better Retrieval Quality $\neq$ Better Retrieval Security

A powerful embedding model can still be vulnerable to optimized retrieval manipulation.

Key Understanding 4: Corpus Competition Matters

The difficulty of retrieval manipulation depends on the corpus.

High Corpus Competition $\Rightarrow$ Harder Attack

Many legitimate documents compete for the top positions.

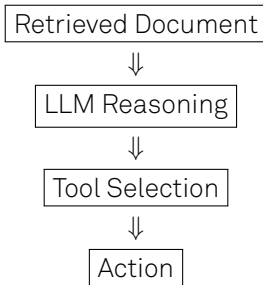
Low Corpus Competition $\Rightarrow$ Easier Attack

Important Insight

Security depends not only on the embedding model but also on the structure and semantic density of the retrieval corpus.

Key Understanding 5: End-to-End Compromise

A retrieval-level attack can propagate through an agent.



Implication

A vulnerability in retrieval can eventually become a complete system-level compromise.

Research Gaps Identified

1. Hybrid retrieval and reranking robustness
2. Cross-model trigger transferability
3. Limited agentic environments
4. Weak retrieval-level defenses
5. Limited multimodal manipulation
6. Positional dependency of triggers
7. Query-specific trigger optimization
8. Email-specific IPI security
9. Lack of provenance-aware retrieval

Most Important Gap

There is still no strong and general-purpose **retrieval-aware defense** against indirect prompt injection.

Research Gap 1: Hybrid Retrieval and Reranking

Current Limitation:

The attack is mainly evaluated against dense embedding retrieval.

Real systems often use:

Dense Retrieval + BM25 + Sparse Retrieval + Reranking

Open Questions:

- Does the attack work against hybrid retrieval?
- Can BM25 reduce attack success?
- Can cross-encoder rerankers detect malicious candidates?

Research Gap 2: Cross-Model Transferability

A trigger optimized for one embedding model may fail on another.

Model A Trigger $\nrightarrow$ Success on Model B

Problem:

A realistic attacker may not know the target embedding model.

Possible Research Direction:

Model-Agnostic Transferable Trigger

Goal:

One Trigger $\rightarrow$ Multiple Models

Research Gap 3: Limited Agentic Environments

The detailed end-to-end evaluation mainly focuses on an email scenario. Other environments remain insufficiently explored:

- Coding assistants
- Software engineering agents
- Enterprise agents
- Cloud infrastructure agents
- Log analysis agents
- Database agents

How does IPI propagate through multi-step agents?

Research Gap 4: Weak Retrieval-Level Defenses

The paper evaluates simple defenses such as:

- Query paraphrasing
- Perplexity filtering
- Token masking

Problem

Adaptive attackers can modify their optimization process and bypass simple defenses.

Need for Robust Retrieval-Aware Defense

The defense should protect the system before malicious documents reach the LLM context.